

SYNOPSIS
ON
"VISTA — VECTOR INTELLIGENT SEMANTIC SEARCH TEXT
ANALYSIS"

Submitted in
Partial Fulfillment of requirements for the Award of Degree

of
Bachelor of Technology
In
Computer Science and Engineering

by
(Project Id: _CS_DS_3C_)

Ansh Gaur (2401641540037)
Aashita Mishra (2401641540004)
Ankit Shukla (2401641540035)
Aditi Sharma (2401641540019)
Anant Dubey (2401641540029)
Arpit Umrao (2401641540049)

Under the supervision of
Hemlata Pant
(Ass. Professor)



Pranveer Singh Institute of Technology.
Kanpur - Agra - Delhi National Highway - 19
Bhauti -Kanpur - 209305.
(Affiliated to Dr. A.P.J. Abdul Kalam Technical University)

1. Introduction

Hospitals and large clinical institutions generate a high volume of unstructured text every day — discharge summaries, radiology notes, lab reports, prescriptions, and progress notes — produced across dozens of departments and specialties. This creates two recurring structural problems. First, terminology drift across departments: the same clinical event is documented differently depending on who wrote it — an ER physician might write "heart attack" where a cardiologist writes "myocardial infarction," and a pathology report might reference "MI" as shorthand — so a keyword search system treats these as three unrelated strings. Second, volume growth outpaces manual organization: folder-based or tag-based filing systems work at small scale but degrade as report volume grows into the thousands, since new conditions, abbreviations, and specialties appear faster than any manual taxonomy can be updated.

VISTA (Vector Intelligent Semantic Search Text Analysis) is a semantic data warehouse for medical text that addresses this problem end-to-end. Instead of storing and matching reports by exact keywords, VISTA converts every report into a numerical vector that encodes its clinical meaning, then organizes and retrieves reports by comparing those vectors. The practical effect is that a query for "myocardial infarction" also surfaces reports that only say "heart attack," because the system compares meaning rather than spelling. The system is built from four cooperating layers — Ingestion, Representation, Storage & Retrieval, and Interface — which together turn a growing, disorganized archive of medical text into a searchable and self-organizing knowledge base for clinicians.

2. Project Objective

- To replace lexical (keyword-based) matching of medical reports with semantic matching, so that clinically equivalent reports written in different terminology are retrieved together.
- To design an end-to-end ingestion pipeline that accepts born-digital text, PDF documents, and scanned images, and normalizes all three into clean text suitable for embedding.
- To generate dense vector embeddings for every report using a pretrained sentence-transformer model, such that clinically similar reports land near each other in vector space.

- To provide fast nearest-neighbor semantic search over the vector index and periodic clustering of the full report set, so reports can be browsed by theme, not only searched by query.
- To keep structured patient metadata (PostgreSQL) architecturally separate from the semantic vector index (Qdrant), so patient-identifying information is never exposed through the similarity-search layer.
- To expose search and cluster-browsing functionality through a FastAPI backend and an interactive Streamlit dashboard usable by non-technical clinical staff.

Overall, the objective is to give clinicians a system where searching for a clinical concept returns every relevant report regardless of who wrote it or which words they used, reducing duplicated documentation and the time spent manually searching through case history.

3. Feasibility Study:

a. Technical Feasibility:

The project is built entirely on mature, well-documented, open-source technologies: a sentence-transformer model (all-MiniLM-L6-v2) from the Sentence-Transformers library for embedding generation, Qdrant as the vector database using HNSW (Hierarchical Navigable Small World graphs) for fast approximate nearest-neighbor search, PostgreSQL for structured metadata, MinIO as an S3-compatible object store for archiving raw source files, and Tesseract/EasyOCR for extracting text from scanned reports. FastAPI serves the backend (chosen for its Pydantic-based validation and native async support) and Streamlit renders the clinician-facing dashboard. All components have active community support, making technical implementation highly feasible for a project of this scope.

b. Operational Feasibility:

The system is designed as a self-contained pipeline — ingest, clean, embed, store, search, cluster — that plugs into an existing hospital's document flow without requiring clinicians to change how they write or file reports. The dashboard's search and cluster-browser views are built for non-technical clinical users, and Docker Compose brings up every service (API, Qdrant, PostgreSQL, MinIO, dashboard) consistently on any host machine, minimizing operational overhead during deployment.

c. Economic Feasibility:

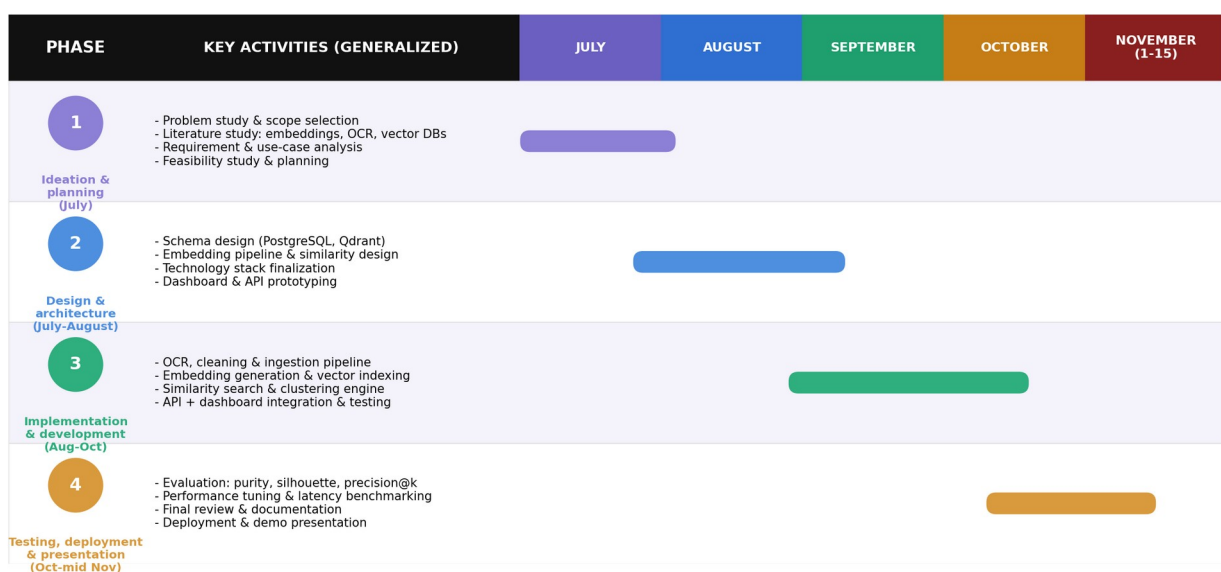
The technology stack is entirely open-source and self-hostable — Qdrant, PostgreSQL, MinIO, Tesseract, FastAPI, and Streamlit all carry no licensing cost. Compute cost is limited to running a small distilled embedding model and standard database services, which keeps the project economically feasible within a typical academic project budget, with cloud scaling only needed if the corpus grows well beyond project scale.

d. Schedule Feasibility:

The project follows a structured, phase-wise roadmap suited to a team of six, moving from ingestion and embedding through to similarity search, clustering, and interface development.

Start Date: 13-July-2026

End Date: 20-Nov-2026



e. Legal Feasibility:

Since VISTA processes medical text that may contain patient-identifying information, privacy is addressed structurally: patient-identifying fields are stored only in PostgreSQL, while the vector index holds only the numerical embedding plus a non-identifying pointer back to the metadata row, so even if the vector store were exposed in isolation it would not by itself reveal patient identity. De-identification at ingestion, access control at the API layer, and audit logging are documented as required practices for any real deployment on genuine patient data.

4. Methodology/ Planning of work

The project follows a structured, phase-wise pipeline across the development roadmap:

Phase 1: Establish foundations — finalize the PostgreSQL metadata schema (reports, patients), set up the Qdrant vector store, and scaffold the FastAPI backend and Docker Compose environment.

Phase 2: Build the ingestion pipeline — accept text, PDF, and scanned-image uploads; branch scanned/image-based input through OCR (Tesseract/EasyOCR) and pass born-digital text straight to cleaning.

Phase 3: Implement the data cleaning stage — remove whitespace and line-break artifacts, strip boilerplate headers/footers, and segment text into sentences before embedding.

Phase 4: Integrate the embedding layer — pass cleaned text through the all-MiniLM-L6-v2 sentence-transformer model to produce one fixed-length vector per report, and store it in Qdrant alongside a metadata pointer into PostgreSQL.

Phase 5: Build the similarity search engine — embed incoming queries the same way and retrieve nearest neighbors from Qdrant using cosine similarity and HNSW-based approximate search.

Phase 6: Build the clustering engine — periodically group the full vector set (K-means and/or hierarchical clustering) so reports can be browsed by theme, validated using silhouette score and cluster purity against the MTSamples medical_specialty label.

Phase 7: Develop the FastAPI endpoints (/search, /clusters/{id}, /ingest) and wire them to the Streamlit dashboard's search view, cluster browser, and report detail view.

Phase 8: Containerize the full stack with Docker Compose (API, Qdrant, PostgreSQL, MinIO, dashboard) for consistent one-command deployment.

Phase 9: Carry out evaluation — clustering purity vs. ground-truth specialty labels, silhouette score, precision@k on hand-labeled query/report pairs, and latency benchmarking at increasing index sizes (5K, 20K, 50K vectors).

Phase 10: Conduct final testing, documentation, and demo preparation, including a live walkthrough of end-to-end semantic search and cluster browsing.

5. Tools/Technology Used:

5.1 Minimum Hardware Requirements

- **CPU:** Quad-Core Processor (Intel i5 / AMD Ryzen 5 or greater)

- **RAM:** 16 GB minimum
- **GPU:** Optional but recommended (for faster embedding generation over large report volumes)
- **HDD/SSD:** 128 GB total storage (SSD preferred)
- **Other:** Reliable internet connection; cloud subscription optional for scaling beyond project scope

5.2 Minimum Software Requirements

- **Operating System:** Windows 11 / Linux
- **Programming Language:** Python 3.11+
- **Backend:** FastAPI
- **Frontend/Dashboard:** Streamlit
- **NLP/Embedding:** Sentence-Transformers (all-MiniLM-L6-v2), Tesseract/EasyOCR
- **Data Layer:** PostgreSQL, Qdrant (vector database), MinIO (object storage)
- **Clustering/Evaluation:** scikit-learn (K-means, hierarchical clustering, silhouette score)
- **DevOps:** Docker and Docker Compose, Git for version control
- **Development Tools:** VS Code / PyCharm

6. References: [IEEE format]:

- a) N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," Proc. EMNLP, 2019.
- b) Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," IEEE Trans. Pattern Analysis and Machine Intelligence, 2020.
- c) R. Smith, "An Overview of the Tesseract OCR Engine," Proc. Ninth Int. Conf. on Document Analysis and Recognition (ICDAR), 2007.
- d) S. Ramírez, "FastAPI," 2018. [Online]. Available: <https://fastapi.tiangolo.com>
- e) MTSamples, "Medical Transcription Sample Reports and Examples." [Online]. Available: <https://www.mtsamples.com>
- f) Qdrant, "Qdrant Vector Database Documentation," 2023. [Online]. Available: <https://qdrant.tech>

Guidelines:

1) Heading Font Size: 14

2) Content Font Size: 12

3) Font Name: Times New Roman

4) Line Spacing: 1.5

5) Alignment: Left & Right both Justified

6) Margin: Normal (Top=2.54 cm, Bottom=2.54 cm, Left=2.54 cm, Right=2.54 cm)

7) Page No: At Bottom Right Side